

Part III

TRANSFORMER

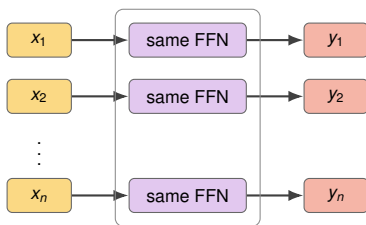
TABLE OF CONTENTS

1 **Origin and needs** 20

2 **Attention Block** 22

3 **Transformer architecture** 23

WHY FEED-FORWARD LAYERS ARE NOT ENOUGH



Token-wise map: $y_i = \text{FFN}(x_i)$
No interaction between different tokens

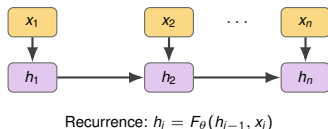
Feed-forward layer

A feed-forward layer maps one vector to one vector: $\text{FFN}(x) = W_2 \phi(W_1 x + b_1) + b_2$.
Applied to tokens, it acts independently: $y_i = \text{FFN}(x_i)$.

Limitation for text

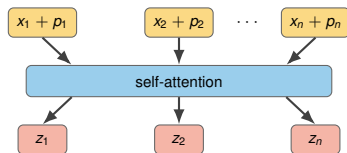
For a sequence $(x_1, \dots, x_n)$, the output y_i depends only on x_i , not on x_j for $j \neq i$. Thus a pure feed-forward layer cannot model word order, context, or long-range dependencies.

FROM RECURRENCE TO CONTEXTUALIZATION



Before attention

Early sequence models encoded token dependence through recurrence: $h_i = F_\theta(h_{i-1}, x_i)$. RNNs, LSTMs, and GRUs process tokens sequentially, so context is carried through the hidden state.



Contextualization: $Z = X + \text{SelfAttn}(X + P)$

Main limitation

Long-range dependencies must pass through many recurrent steps. This makes training sequential, hard to parallelize, and can compress or forget distant information.

Transformer idea

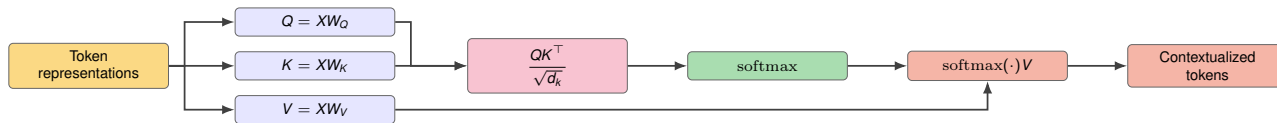
Transformers contextualize all tokens in parallel. With positional information P , self-attention forms $Z = X + \text{SelfAttn}(X + P)$, so each token can directly use information from the other tokens.

ATTENTION BLOCK

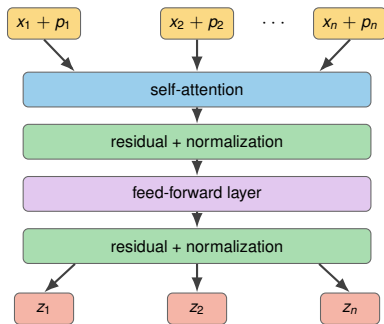
Self-attention mechanism

Each token produces a *query*, a *key*, and a *value*. Queries are matched with keys to form attention weights, which are then used to average the values:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^{\top}}{\sqrt{d_k}}\right) V.$$



WHAT IS A TRANSFORMER?



$Z = \text{Block}(X + P)$, where attention mixes tokens and FFN transforms each token

Core idea

A Transformer is a sequence model that repeatedly contextualizes tokens. Each token representation is updated using information from the other tokens.

One Transformer block

Starting from token embeddings $X = (x_1, \dots, x_n)$ and positional embeddings P , a block applies self-attention and a feed-forward layer: $H = X + \text{SelfAttn}(X + P)$, then $Z = H + \text{FFN}(H)$.

Intuition

Self-attention mixes information across tokens. The feed-forward layer then transforms each contextualized token independently. Stacking many blocks gives increasingly rich contextual representations.